# CARDSCOUT AI

**A Project report**

Submitted in partial fulfilment of the

Requirements for the award of the Degree of

**BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)**

**By**

Mr.Sahil Sanjay Kadam

**Under the esteemed guidance of**

**Ms.Kajal Jaisinghani**

(Head, Dept of I.T)

**DEPARTMENT OF INFORMATION TECHNOLOGY**

## B. K. BIRLA COLLEGE, KALYAN

**(EMPOWERED AUTONOMOUS STATUS)**

***(Affiliated to University of Mumbai)***

**KALYAN, 421304 MAHARASHTRA 2024-2025**

## PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

***(Note:All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)***

PNR No.: **……** **Roll no:** _____

1. Name of the Student

_____

2. Title of the Project

_____

3. Name of the Guide

_____

4. Teaching experience of the Guide _____

5. Is this your first submission? Yes _____ No _____

Signature of the Student
Date: ……

Signature of the Guide
Date: ……

Signature of the Coordinator
Date: ……

# B. K. BIRLA COLLEGE, KALYAN

(EMPOWERED AUTONOMOUS STATUS)

*(Affiliated to University of Mumbai)*

**KALYAN, 421304 MAHARASHTRA**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

## CERTIFICATE

This is to certify that the project entitled**, "Cardscout AI"**, is bonafied work of **Mr. Sahil Sanjay Kadam** bearing Seat. No. **5144985** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from the University of Mumbai.

**Internal Guide** **Coordinator**

**External Examiner**

**Date:** **College seal**

# ABSTRACT

Selecting the right credit card in India is difficult. Consumers face hundreds of products, complex reward structures, changing fee rules and hidden penalty clauses. Most existing tools are static affiliate websites that recommend cards without considering an individual's actual spending, credit limits or existing cards. As a result, users miss rewards, fail to reach fee-waiver targets and sometimes hurt their CIBIL score through high credit utilization.

**CardScout AI** is a Generative AI-powered financial advisory platform designed to address this by providing hyper-personalized, real-time credit card optimization. Built with Python, Streamlit, and a PostgreSQL backend, the system chains Google's Gemini Large Language Model with the Tavily Search API to ground its financial recommendations in live market data.

This project, CardScout AI, is a generative-AI-driven financial advisory platform that addresses these problems. It collects a user's profile and monthly spending blueprint, combines it with live web data retrieved through the Tavily Search API, and uses Google's Gemini Large Language Model to generate a personalized Top-5 credit card roadmap. After acquisition, users manage their cards in the Obsidian Vault, a secure, PostgreSQL-backed dashboard containing seven analytical engines: Fee Waiver Pacing, Live Offers Radar, Milestone Rewards, Subscription Saver, Forex Engine, Point Valuation Matrix and Penalty Audit. A Battleground tool compares any card against the AI's top pick, and a Co-Pilot advisor issues a SWIPE or HOLD verdict on a planned expense, enforcing a 30% credit-utilization rule.

The system is built in Python 3.11 with a Streamlit front end and custom CSS, PostgreSQL (Supabase) for storage, PBKDF2-HMAC-SHA256 salted password hashing for security, an API-key rotation and model-fallback mechanism for high availability, and FPDF for report export.

**Keywords:** Generative AI, Gemini, Tavily Search API, Credit Card Optimization, Streamlit, PostgreSQL, Prompt Engineering, Financial Literacy.

# ACKNOWLEDGEMENT

In the journey towards success, dedication, hard work, patience, and expert guidance serve as the fundamental pillars of our efforts. Our college, B. K. Birla College of Arts, Science & Commerce (Empowered Autonomous Status), has provided a highly nurturing environment for learning and professional growth.

A special tribute is reserved for my esteemed project guide, **Kajal Jaisinghani** (Asst.Professor, Department of Computer Science), whose meticulous guidance, continuous encouragement, and attention to detail have been invaluable throughout the development of CardScout AI. Their technical insights and mentorship were instrumental in shaping the architecture and execution of this project.

I would also like to express my sincere thanks to the faculty members of the Computer Science department for their foundational support and for equipping me with the essential skills required to undertake a project of this technical complexity.

Finally, I extend my heartfelt gratitude to my family, friends, and everyone who has supported me, collectively transforming my aspirations into tangible achievements and illuminating the path forward.

# DECLARATION

I hereby declare that the project entitled, "CARDSCOUT AI" done at Kalyan, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of the degree of BACHELOR OF SCIENCE (DATA SCIENCE) to be submitted as final semester project as part of our curriculum.

[SAHIL SANJAY KADAM]

# TABLE OF CONTENTS

**Chapter 1: Introduction.....10**

1.1 Background .....10

1.2 Objectives .....11

1.3 Purpose, Scope and Applicability .....12

1.3.1 Purpose ..... 12

1.3.1 Scope ..... 12

1.3.2 Applicability ..... 12

1.4 Achievements.....11

1.5 Organisation of Report.....11

**Chapter 2: Survey of Technologies .....13**

2.1 Available Technologies and Comparative Study…….....11

2.2 Justification of Selected Technologies.....11

**Chapter 3: Requirements and Analysis.....23**

3.1 Problem Definition.....10

3.2 Requirements Specification.....10

3.3 Planning and Scheduling.....10

3.4 Software and Hardware Requirements.....10

3.5 Preliminary Product Description.....10

3.6 Conceptual Models.....10

**Chapter 4: System Design .....26**

4.1 Basic Modules.....26-29

4.2 Data Design.....30-31

4.2.1 Schema Design.....30

4.2.2 Data Integrity and Constraints.....31

4.3 Procedural Design.....30-31

4.3.1 Logic Diagrams.....30

4.3.2 Data Structures.....30

4.3.3 Algorithms Design.....30

4.4 User Interface Design.....30-31

4.5 Security Issues.....30-31

4.6 Test Cases Design.....30-31

**Chapter 5: Implementation and Testing.....32**

5.1 Implementation Approaches.....32-33

5.2 Coding Details and Code Efficiency.....33-36

5.2.1 Code Efficiency.....30

5.3 Testing Approach.....37-38

5.3.1 Unit Testing.....30

5.3.2 Integrated Testing.....30

5.3.3 Beta Testing.....30

5.4 Modifications and Improvements.....39-40

5.5 Test Cases.....39-40

**Chapter 6: Results and Discussion.....41**

6.1 Test Reports.....41

6.2 User Documentation.....41

**Chapter 7: Conclusions.....42**

7.1 Conclusion.....39-40

7.1.1 Significance of the System.....30

7.2 Limitations of the System.....39-40

7.3 Future Scope of the Project.....39-40

**References .....42**

**Glossary.....42**

**Appendix A.....42**

**Appendix B.....42**

# TABLE OF FIGURES

Fig 1: System Architecture..... 20 \

Fig 2: Guest Scout ..... 20

Fig 3: Expense Blueprint..... 20

Fig 4: Intelligence Hub..... 20

Fig 5: Battleground..... 20

Fig 6: Vault Login..... 20

Fig 7: Digital Wallet..... 20

Fig 8: Fee Waiver Pacing..... 20

Fig 9: Live Offers Radar..... 20

Fig 10: Milestone Rewards..... 20

Fig 11: Subscription Saver..... 20

Fig 12: Forex Engine..... 20

Fig 13: Point Valuation Matrix..... 20

Fig 14: Penalty Audit..... 20

Fig 15: PDF Report..... 20

# Chapter 1: Introduction

**CardScout AI** is a comprehensive, Generative AI-driven financial advisory web application that helps Indian consumers choose, use, and manage credit cards intelligently. By replicating the logical workflow of a human financial advisor, it studies a user's income, granular spending habits, credit score band, and existing banking relationships to generate personalized, data-backed financial roadmaps. Moving beyond static recommendations, CardScout AI provides a secure, post-acquisition command center—the Obsidian Vault—which features seven distinct analytical engines to actively track fee-waiver progress, live market offers, reward-point arbitrage, forex markups, and hidden penalties.

## 1.1 Background

In the modern financial landscape, credit card usage in India has grown exponentially. Consumers are overwhelmed by the sheer volume of credit card options, complex reward matrices, dynamic fee structures, and hidden penalty clauses buried deep within terms and conditions. A significant lack of financial literacy leads consumers to select sub-optimal credit products based on brand familiarity rather than mathematical fit.

Existing comparison websites are predominantly static, affiliate-driven portals. These platforms fail to provide hyper-personalized advice because they do not account for a user's actual spending blueprint or existing digital wallet inventory. Furthermore, manual tools like spreadsheets are error-prone and quickly become outdated. Recent advancements in Large Language Models (LLMs), combined with live web search APIs, make it possible to build an intelligent, retrieval-augmented advisor that reasons over up-to-date market information to provide dynamic financial guidance.

## 1.2 Objectives

The overarching goal of the CardScout AI project is to revolutionize how consumers interact with credit products by transitioning from static, affiliate-driven comparison websites to a dynamic, hyper-personalized financial advisory ecosystem. To achieve this, the project is driven by four primary, highly technical objectives.

**1.Development of a Hyper-Personalized Recommendation Engine** The first major objective is to build an intelligent profiling wizard that moves beyond generic credit card suggestions. Traditional platforms recommend cards based on broad categories, which often leads to sub-optimal financial yields for the user. CardScout AI aims to ingest a highly granular financial demographic profile—capturing exact monthly expenditures across specific verticals such as Forex, Quick Commerce, Utilities, and Lifestyle. By cross-referencing this exact spending blueprint with existing banking relationships and preferred reward mechanisms (e.g., direct cashback vs. air miles), the system's objective is to mathematically calculate and generate a Top-5 Credit Card Roadmap. This roadmap must not only rank the cards by maximum achievable yield but also provide a step-by-step, actionable acquisition strategy tailored specifically to the individual user. To implement a live data grounding mechanism by chaining the Tavily Web Search API with Google's Gemini LLM, ensuring responses rely on current market data.

**2. Implementation of a Live Data Grounding Mechanism (RAG Architecture)** A critical limitation of traditional Large Language Models (LLMs) in the financial sector is their reliance on static training data, which leads to hallucinations and outdated advice regarding ever-changing banking fees, interest rates, and live market offers. To solve this, a core objective is to implement a Retrieval-Augmented Generation (RAG) architecture. By programmatically chaining the Tavily Web Search API with Google's Gemini LLM, the system is designed to fetch real-time, live internet data at the exact moment of the user's query. This ensures that every piece of advice, penalty audit, and reward valuation generated by the AI is strictly grounded in current, verifiable market data rather than obsolete historical patterns. To enforce a strict 30% credit-utilization rule via a conversational Co-Pilot assistant, protecting users from inadvertently damaging their CIBIL scores.

**3. Construction of a Secure, Deterministic Portfolio Management System** Credit card optimization does not end at acquisition; post-acquisition management is where most consumers lose money to hidden penalties and

missed rewards. Therefore, a primary objective is to engineer the "Obsidian Vault"—a robust, PostgreSQL-backed dashboard designed for secure portfolio management. The objective is to build seven distinct, deterministic analytical engines within this vault (Fee Waiver Pacing, Live Offers Radar, Milestone Rewards, Subscription Saver, Forex Engine, Point Valuation Matrix, and Penalty Audit). While the LLM is used to extract unstructured data from complex banking terms, these seven engines must utilize hard-coded, deterministic Python mathematics to calculate exact metrics (like GST on forex markups and point-value arbitrage). This hybrid approach ensures absolute mathematical accuracy while leveraging AI for deep data extraction.

**4. Active Protection of Credit Health via a Conversational Co-Pilot**
Financial literacy and credit score protection are at the heart of this project. High credit utilization is one of the leading causes of CIBIL score degradation among young professionals and students. A vital objective of CardScout AI is to actively protect the user's credit health by enforcing a strict 30% credit-utilization rule. To accomplish this, the project aims to develop the "CardScout Co-Pilot"—a conversational AI assistant equipped with the context of the user's saved digital wallet and credit limits. When a user inputs an impromptu or planned expense, the Co-Pilot's objective is to instantly evaluate the transaction against the 30% threshold, issuing either a green "SWIPE" recommendation with the best card to use, or a strict red "HOLD" advisory warning the user of potential CIBIL damage, thus enforcing responsible credit behavior in real-time.

## 1.3 Purpose, Scope, and Applicability

### 1.3.1 Purpose

The purpose of CardScout AI is to bridge the gap between complex banking products and consumer financial literacy. It aims to democratize access to hyper-personalized, data-backed credit card advice that is typically only available from paid financial advisors. The project serves to help users maximize their financial yields (cashback and travel miles), avoid predatory financial penalties, and actively protect their credit scores.

### 1.3.2 Scope

The scope of this project encompasses the development of a full-stack Python web application. The main functional components include:

- **Guest Scout:** A multi-step profiling wizard that captures financial demographics without requiring a login.
- **Intelligence Hub:** Generates the AI roadmap, approval-odds indicators, and peer-to-peer "Battleground" card comparisons.
- **Obsidian Vault:** An authenticated, PostgreSQL-backed dashboard housing a Digital Wallet, subscription tracker, and seven analytical engines.
- **CardScout Co-Pilot:** A real-time conversational advisor providing SWIPE/HOLD verdicts for planned expenses.
- **Methodology & Limitations:** The system employs deterministic Python mathematics alongside Generative AI data extraction. The scope is strictly advisory; the application does not apply for cards, connect to live bank accounts, or process financial payments.

### 1.3.3 Applicability

- **Direct Application:** Highly applicable to salaried individuals, business owners, and students seeking to choose their first or next credit card mathematically.
- **Indirect Application:** Existing cardholders can utilize the platform to optimize recurring digital subscriptions, track annual fee waivers, and avoid heavy forex markups during international travel.
- **Broader Impact:** Serves as a reusable educational template demonstrating the practical implementation of Retrieval-Augmented Generation (RAG) and prompt engineering in the financial technology sector.

## 1.4 Achievements

Upon successful completion of the project, several significant technical and domain-specific milestones were achieved:

- Successfully implemented a robust Retrieval-Augmented Generation (RAG) architecture, bridging live internet scraping (Tavily) with advanced language generation (Gemini).
- Engineered a secure, relational database backend using PostgreSQL on Supabase, successfully handling user authentication via PBKDF2-HMAC-SHA256 salted hashing and preventing SQL injection through parameterized queries.
- Developed complex deterministic financial algorithms in Python to accurately calculate utilization ratios, forex GST penalties, and reward point arbitrage, eliminating the arithmetic hallucination risks common in LLMs.
- Gained profound practical expertise in full-stack Python development using the Streamlit framework, session state management, API key rotation logic, and dynamic PDF report generation.

## 1.5 Organisation of Report

The remainder of this project report is structured as follows:

- **Chapter 2 (Survey of Technologies):** Presents a comparative study and detailed justification of the open-source technologies, programming languages, and AI models selected for this project.
- **Chapter 3 (Requirements and Analysis):** Defines the core problem, outlines functional and non-functional requirements, details software and hardware constraints, and presents conceptual models.
- **Chapter 4 (System Design):** Describes the system architecture, procedural design, database schemas, and data integrity constraints utilized to build the platform.
- **Chapter 5 (Implementation and Testing):** Discusses the coding approaches, algorithmic efficiency, and the methodologies used for unit and integration testing.
- **Chapter 6 (Results and Discussion):** Provides comprehensive test reports alongside detailed user documentation, featuring screenshots of the application interfaces.
- **Chapter 7 (Conclusions):** Summarizes the significance of the developed system, highlights its current limitations, and outlines the future scope for subsequent project iterations.

# CHAPTER 2: SURVEY OF TECHNOLOGIES

Developing a highly dynamic, generative AI-driven financial platform like CardScout AI requires integrating real-time web scraping, complex deterministic backend logic, strict relational database management, and interactive frontend visualizations. This chapter presents a comparative study of the available technologies in these domains and provides a detailed justification for the final technology stack selected to execute the project's objectives.

## 2.1 Available Technologies and Comparative Study

**1. Frontend and Web Frameworks**

- **React.js (MERN Stack):** A powerful JavaScript library for building highly dynamic single-page applications. While excellent for massive enterprise applications, React requires building a separate backend API in Node.js. This significantly increases development overhead and context-switching for data-heavy, analytical Python-based applications.
- **Django and Flask:** Traditional Python web frameworks. Django provides a massive, "batteries-included" ecosystem, while Flask offers a lightweight routing mechanism. However, both require managing separate HTML, CSS, and JavaScript files, slowing down the rapid prototyping of AI-driven tools.
- **Streamlit:** An open-source Python framework specifically built for Machine Learning and Data Science teams. It allows developers to build interactive web applications entirely in Python without needing separate frontend languages, making it highly efficient for AI-integrated dashboards and state-driven multi-step wizards.

**2. Database Management Systems (DBMS)**

- **MongoDB:** A popular NoSQL database that stores data in flexible, JSON-like documents. While highly scalable for unstructured data, it lacks the strict relational integrity (such as foreign keys and cascade rules) required for managing structured financial data, like linking specific user profiles to their digital wallets and recurring subscriptions.
- **MySQL:** A widely used traditional Relational Database Management System (RDBMS). It is reliable but can be complex to set up and scale rapidly in cloud-native environments without dedicated DevOps resources.
- **PostgreSQL:** An advanced, open-source object-relational database known for its robustness, complex query optimization, and strict data integrity constraints. Managed cloud-hosted solutions like Supabase provide PostgreSQL with built-in connection pooling, making it ideal for scalable applications.

**3. Generative AI and Large Language Models (LLMs)**

- **OpenAI (GPT-4 / GPT-3.5):** Industry-leading models with excellent reasoning capabilities. However, API costs can be prohibitive for high-throughput, free-to-use consumer applications that require rapid, repeated calls.
- **Anthropic (Claude):** Known for its large context windows and safe outputs, but typically slower in response times for real-time conversational agents like the CardScout Co-Pilot.
- **Google Generative AI (Gemini):** Google's state-of-the-art multimodal AI models. The Gemini Flash-Lite series (gemini-3.5-flash-lite) is specifically optimized for high-volume, low-latency tasks. It is the most cost-effective model for structured prompt engineering, rapid data extraction, and seamless failover handling.

**4. Real-Time Web Search and Scraping APIs**

- **BeautifulSoup & Selenium:** Traditional Python libraries for web scraping. They require constant maintenance as website layouts change and are easily blocked by anti-bot protections on banking websites.
- **Google Custom Search API:** Returns raw search engine results pages that require heavy post-processing to extract readable text suitable for an LLM.
- **Tavily Search API:** An AI-optimized search engine that actively scours the web and returns clean, synthesized, and context-rich text ready to be injected directly into a Large Language Model's prompt, making it ideal for a Retrieval-Augmented Generation (RAG) architecture.

## 2.2 Justification of Selected Technologies

After analyzing the project's requirement for a Retrieval-Augmented Generation (RAG) architecture and deterministic mathematical calculations, the following technology stack was selected and implemented:

- **Programming Language - Python 3.11:** Python was selected as the core language because it provides a mature ecosystem for AI integration (google-generativeai), data manipulation (pandas), and database connectivity (psycopg2-binary).
- **Frontend - Streamlit:** Streamlit was chosen because it allows for rapid, stateless frontend development directly in Python. It handles session state management required for the Guest Scout wizard and offers built-in caching (st.cache_data) with a time-to-live parameter, which is critical for caching AI and search results to preserve API quotas. Custom CSS was injected to provide a modern, dark-themed frosted-glass interface.
- **Backend & Database - PostgreSQL on Supabase:** PostgreSQL was selected for its strict relational integrity, allowing secure one-to-many linkages between the users, wallet, and subscriptions tables. Supabase was chosen as the hosting provider because it offers managed PostgreSQL instances with connection URLs that integrate seamlessly via psycopg2.
- **AI Engine - Google Generative AI (Gemini):** The Gemini LLM was chosen for its unparalleled speed in processing structured prompts. Specifically, the gemini-3.5-flash-lite and gemini-3.1-flash-lite models were integrated as fallbacks to ensure high availability and rapid data extraction for the Obsidian Vault's seven analytical engines.
- **Live Data Fetching - Tavily Web Search API:** To ensure the AI's advice is grounded in current market reality (e.g., live sales, current forex markups), the tavily-python library was utilized. It bypasses the hallucination risks of static LLMs by fetching live data that is fed directly into Gemini.
- **Security - Hashlib (PBKDF2-HMAC-SHA256):** To secure user authentication, the native Python hashlib library was selected to implement PBKDF2-HMAC-SHA256 password hashing. By generating a unique 16-byte random salt per user and iterating the hash 100,000 times, the system provides enterprise-grade protection.
- **Report Generation - FPDF:** The fpdf library was chosen over heavier alternatives because it allows for dynamic, programmatic generation of the personalized PDF roadmaps, including custom table rendering and UTF-8 sanitization, directly from the Streamlit UI.

# CHAPTER 3: REQUIREMENTS AND ANALYSIS

## 3.1 Problem Definition

In the modern financial landscape, consumers are overwhelmed by the sheer volume of credit card options, complex reward matrices, dynamic fee structures, and hidden penalty clauses. A significant lack of financial literacy leads to consumers selecting sub-optimal credit products based on brand familiarity rather than mathematical suitability. Consequently, users miss out on potential financial yield (such as cashback and travel miles), fail to meet fee-waiver pacing targets, and inadvertently damage their CIBIL scores due to high credit utilization (exceeding the recommended 30% threshold).

Existing solutions in the market are largely static, affiliate-driven comparison websites. These platforms fail to provide hyper-personalized, real-time advice because they do not account for an individual's exact spending habits, existing digital wallet inventory, or real-time banking offers. The overall problem is the lack of an intelligent, dynamic, and data-grounded advisory system capable of optimizing a consumer's credit portfolio both before and after card acquisition.

## 3.2 Requirements Specification

The requirements specification outlines the capabilities the system must possess to solve the defined problem, independent of the underlying implementation technologies.

**1. Functional Requirements**

- **User Management:** The system must allow users to create accounts, authenticate securely, and manage their sessions.
- **Profile Capture (Guest Scout):** The system must capture granular demographic and financial data, including monthly spending across categories (Forex, Utilities, Lifestyle, etc.), credit score bands, and reward preferences.
- **Roadmap Generation:** The system must synthesize user data with live market data to generate a personalized Top-5 credit card recommendation and a step-by-step acquisition plan.
- **Digital Wallet Management:** Authenticated users must be able to add, update, and remove credit cards and their respective credit limits within a secure database.
- **Analytical Engines:** The system must provide specific engines to calculate fee waiver pacing, extract live offers, track milestone rewards, optimize recurring subscriptions, calculate forex penalties, compute reward point arbitrage, and audit hidden penalties.
- **Co-Pilot Advisor:** A conversational interface must evaluate impromptu expenses against the user's saved card limits and issue a strict SWIPE/HOLD verdict based on a 30% utilization rule.
- **Report Export:** The system must generate a downloadable, formatted PDF report of the financial roadmap.

**2. Non-Functional Requirements**

- **Security:** Passwords must be hashed using a cryptographically secure algorithm (PBKDF2-HMAC-SHA256) with unique per-user salting.
- **Performance and Reliability:** The system must maintain high availability through automated API key rotation, fallback AI models, and query caching to prevent rate-limiting.

- **Usability:** The interface must be intuitive, featuring a step-by-step wizard and a clear, color-coded verdict system (Green/Amber/Red).

## 3.3 Planning and Scheduling

Planning and scheduling are critical components of the software development life cycle (SDLC) to ensure timely completion of the CardScout AI project. The project was divided into manageable milestones: Requirement Gathering, UI/UX Design, Database Schema Formulation, AI/API Integration, System Testing, and Deployment.

Constraints such as API quota limits and database connection pooling were factored into the development schedule. To visualize the project timeline and resource allocation, a Gantt Chart was utilized. Furthermore, a Program Evaluation Review Technique (PERT) chart was developed to identify the critical path of tasks, ensuring that dependencies—such as establishing the PostgreSQL connection before building the Obsidian Vault—were strictly followed.

## 3.4 Software and Hardware Requirements

The development and implementation of CardScout AI require the following environments:

**Hardware Requirements:**

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher (for development).
- **RAM:** Minimum 8 GB (16 GB recommended for running local virtual environments).
- **Storage:** SSD with at least 256 GB of available space.
- **Network:** High-speed broadband/Wi-Fi connection (mandatory for real-time communication with Gemini and Tavily APIs).
- **Client Device:** Any desktop, laptop, or mobile device with a modern web browser.

**Software Requirements:**

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Programming Language:** Python 3.11.
- **Frontend Framework:** Streamlit (with custom CSS).
- **Database Management:** PostgreSQL (hosted on Supabase) accessed via psycopg2.
- **AI and APIs:** google-generativeai (Gemini API), tavily-python (Web Search API).
- **Additional Libraries:** pandas (data manipulation), hashlib (security), fpdf (PDF generation), python-dotenv (environment configuration).
- **IDE and Version Control:** Visual Studio Code (VS Code), Git, and GitHub.

## 3.5 Preliminary Product Description

CardScout AI is a holistic, Generative AI-powered financial advisory platform. The system operates across two primary phases: Pre-Acquisition and Post-Acquisition. In the Pre-Acquisition phase, the **Guest Scout** module captures the user's spending blueprint and interfaces with the **Intelligence Hub** to deliver a data-grounded credit card roadmap, complete with approval odds and peer-to-peer card comparisons.

In the Post-Acquisition phase, the system transitions into the **Obsidian Vault**. This secure, authenticated dashboard allows users to manage their digital wallets and recurring subscriptions. It executes seven specialized analytical engines that combine real-time AI web scraping with deterministic Python mathematics to optimize fee waivers, forex markups, and reward redemptions. Ultimately, the product serves to automate complex financial analysis, delivering elite-level advisory services directly to the consumer.

## 3.6 Conceptual Models

To understand the problem domain and the sequence of allowable operations, conceptual models were developed.

**1. Entity-Relationship (ER) Model:** The database is structured around a central users entity containing authentication credentials. This entity shares a one-to-many relationship with the wallet entity (storing saved cards and limits) and the subscriptions entity (storing recurring bills linked to specific cards). Strict foreign key constraints enforce referential integrity across these entities.

**2. Data Flow Diagram (DFD):** The logical flow of information begins with user input via the Streamlit frontend. This data is passed to the AI Agent (Python core), which constructs a search query sent to the Tavily API. The retrieved live web context, combined with the user profile, is injected into a structured prompt sent to the Gemini API. The AI's generated insights flow back to the Python backend, where deterministic calculations are applied, and finally, the output is rendered on the UI or exported via the FPDF module

# CHAPTER 3: REQUIREMENTS AND ANALYSIS

## 3.1 Problem Definition

In the modern financial landscape, consumers are overwhelmed by the sheer volume of credit card options, complex reward matrices, dynamic fee structures, and hidden penalty clauses. A significant lack of financial literacy leads to consumers selecting sub-optimal credit products based on brand familiarity rather than mathematical suitability. Consequently, users miss out on potential financial yield (such as cashback and travel miles), fail to meet fee-waiver pacing targets, and inadvertently damage their CIBIL scores due to high credit utilization (exceeding the recommended 30% threshold).

Existing solutions in the market are largely static, affiliate-driven comparison websites. These platforms fail to provide hyper-personalized, real-time advice because they do not account for an individual's exact spending habits, existing digital wallet inventory, or real-time banking offers. The overall problem is the lack of an intelligent, dynamic, and data-grounded advisory system capable of optimizing a consumer’s credit portfolio both before and after card acquisition.

### 3.2 Requirements Specification

The requirements specification outlines the capabilities the system must possess to solve the defined problem, independent of the underlying implementation technologies.

**1. Functional Requirements**

- **User Management:** The system must allow users to create accounts, authenticate securely, and manage their sessions.
- **Profile Capture (Guest Scout):** The system must capture granular demographic and financial data, including monthly spending across categories (Forex, Utilities, Lifestyle, etc.), credit score bands, and reward preferences.
- **Roadmap Generation:** The system must synthesize user data with live market data to generate a personalized Top-5 credit card recommendation and a step-by-step acquisition plan.
- **Digital Wallet Management:** Authenticated users must be able to add, update, and remove credit cards and their respective credit limits within a secure database.
- **Analytical Engines:** The system must provide specific engines to calculate fee waiver pacing, extract live offers, track milestone rewards, optimize recurring subscriptions, calculate forex penalties, compute reward point arbitrage, and audit hidden penalties.
- **Co-Pilot Advisor:** A conversational interface must evaluate impromptu expenses against the user's saved card limits and issue a strict SWIPE/HOLD verdict based on a 30% utilization rule.
- **Report Export:** The system must generate a downloadable, formatted PDF report of the financial roadmap.

**2. Non-Functional Requirements**

- **Security:** Passwords must be hashed using a cryptographically secure algorithm (PBKDF2-HMAC-SHA256) with unique per-user salting.
- **Performance and Reliability:** The system must maintain high availability through automated API key rotation, fallback AI models, and query caching to prevent rate-limiting.

- **Usability:** The interface must be intuitive, featuring a step-by-step wizard and a clear, color-coded verdict system (Green/Amber/Red).

**3.3 Planning and Scheduling**

Planning and scheduling are critical components of the software development life cycle (SDLC) to ensure timely completion of the CardScout AI project. The project was divided into manageable milestones: Requirement Gathering, UI/UX Design, Database Schema Formulation, AI/API Integration, System Testing, and Deployment.

Constraints such as API quota limits and database connection pooling were factored into the development schedule. To visualize the project timeline and resource allocation, a Gantt Chart was utilized. Furthermore, a Program Evaluation Review Technique (PERT) chart was developed to identify the critical path of tasks, ensuring that dependencies—such as establishing the PostgreSQL connection before building the Obsidian Vault—were strictly followed.

*(Note to student: Insert your Gantt Chart and PERT Chart images here as required by the college)*

- **[Insert Gantt Chart Image Here]**
- **[Insert PERT Chart Image Here]**

**3.4 Software and Hardware Requirements**

The development and implementation of CardScout AI require the following environments:

**Hardware Requirements:**

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher (for development).
- **RAM:** Minimum 8 GB (16 GB recommended for running local virtual environments).
- **Storage:** SSD with at least 256 GB of available space.
- **Network:** High-speed broadband/Wi-Fi connection (mandatory for real-time communication with Gemini and Tavily APIs).
- **Client Device:** Any desktop, laptop, or mobile device with a modern web browser.

**Software Requirements:**

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Programming Language:** Python 3.11.
- **Frontend Framework:** Streamlit (with custom CSS).
- **Database Management:** PostgreSQL (hosted on Supabase) accessed via psycopg2.
- **AI and APIs:** google-generativeai (Gemini API), tavily-python (Web Search API).
- **Additional Libraries:** pandas (data manipulation), hashlib (security), fpdf (PDF generation), python-dotenv (environment configuration).
- **IDE and Version Control:** Visual Studio Code (VS Code), Git, and GitHub.

**3.5 Preliminary Product Description**

CardScout AI is a holistic, Generative AI-powered financial advisory platform. The system operates across two primary phases: Pre-Acquisition and Post-Acquisition. In the Pre-Acquisition phase, the **Guest Scout** module captures the user's spending blueprint and interfaces with the **Intelligence Hub** to deliver a data-grounded credit card roadmap, complete with approval odds and peer-to-peer card comparisons.

In the Post-Acquisition phase, the system transitions into the **Obsidian Vault**. This secure, authenticated dashboard allows users to manage their digital wallets and recurring subscriptions. It executes seven specialized analytical engines that combine real-time AI web scraping with deterministic Python mathematics to optimize fee waivers, forex markups, and reward redemptions. Ultimately, the product serves to automate complex financial analysis, delivering elite-level advisory services directly to the consumer.

### 3.6 Conceptual Models

To understand the problem domain and the sequence of allowable operations, conceptual models were developed.

**1. Entity-Relationship (ER) Model:** The database is structured around a central users entity containing authentication credentials. This entity shares a one-to-many relationship with the wallet entity (storing saved cards and limits) and the subscriptions entity (storing recurring bills linked to specific cards). Strict foreign key constraints enforce referential integrity across these entities.

**2. Data Flow Diagram (DFD):** The logical flow of information begins with user input via the Streamlit frontend. This data is passed to the AI Agent (Python core), which constructs a search query sent to the Tavily API. The retrieved live web context, combined with the user profile, is injected into a structured prompt sent to the Gemini API. The AI's generated insights flow back to the Python backend, where deterministic calculations are applied, and finally, the output is rendered on the UI or exported via the FPDF module

# CHAPTER 4: SYSTEM DESIGN

System design translates the requirements gathered in the previous phase into a blueprint for constructing the software. This chapter describes the desired features and operations in detail, encompassing module division, database schema, procedural logic, interface design, security protocols, and test case planning.

## 4.1 Basic Modules

Following the divide-and-conquer approach, CardScout AI is divided into several highly cohesive, loosely coupled modules that are developed independently and integrated into a unified platform:

- **Guest Scout (Profiling Module):** A state-driven wizard that captures a user's financial demographics and granular spending blueprint across specific categories (Forex, Lifestyle, Utilities, etc.).
- **Intelligence Hub (AI Roadmap Module):** Integrates with the AI layer to generate a mathematically optimized Top-5 credit card roadmap, calculate approval odds, and host "The Battleground" for peer-to-peer card comparisons.
- **Obsidian Vault (Portfolio Management Module):** The secure, authenticated dashboard for post-acquisition management. It contains the user's Digital Wallet and seven deterministic analytical engines: Fee Waiver Pacing, Live Offers Radar, Milestone Rewards, Subscription Saver, Forex Engine, Point Valuation Matrix, and Penalty Audit.
- **CardScout Co-Pilot (Conversational Module):** A real-time advisory module that evaluates impromptu expenses against saved wallet limits, issuing strict SWIPE/HOLD verdicts to enforce a 30% utilization rule.
- **Export and Sharing Module:** Utilizes the FPDF library to dynamically sanitize and render the AI-generated Markdown roadmap into a formatted, downloadable PDF.

## 4.2 Data Design

Data design determines how information is organized, managed, and manipulated within the application.

### 4.2.1 Schema Design

The backend relies on a PostgreSQL relational database hosted on Supabase. The database schema is dynamically initialized and consists of three primary tables:

- **users Table:** Stores authentication data. Columns include id (SERIAL), username (TEXT), password_hash (TEXT), and salt (TEXT).
- **wallet Table:** Stores the user's digital credit card inventory. Columns include id (SERIAL), username (TEXT), card_name (TEXT), and card_limit (REAL).
- **subscriptions Table:** Tracks recurring digital auto-debits. Columns include id (SERIAL), username (TEXT), service_name (TEXT), monthly_cost (REAL), and card_used (TEXT).

### 4.2.2 Data Integrity and Constraints

To maintain absolute data integrity, strict validity checks and database constraints are applied:

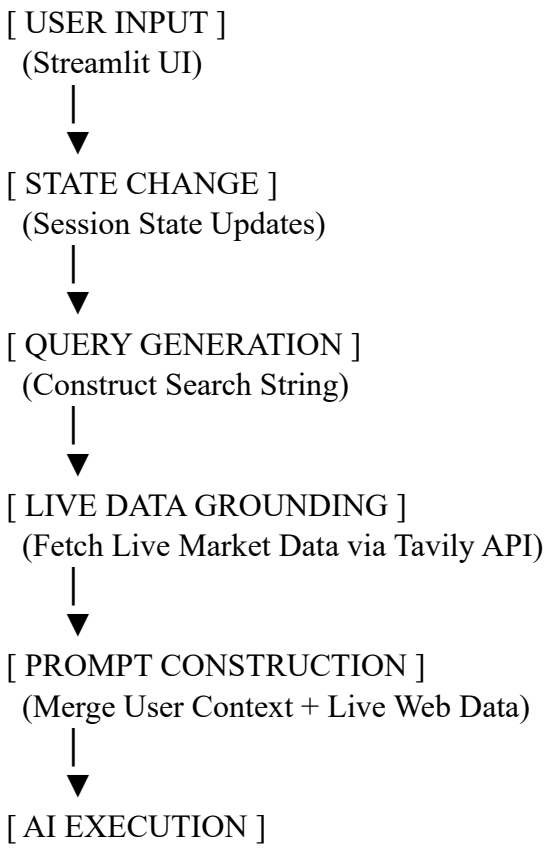
- **Primary Keys:** The id column in every table is designated as a SERIAL PRIMARY KEY for unique row identification.
- **Unique Constraints:** The username in the users table enforces a UNIQUE constraint. Attempts to register duplicate IDs are intercepted by catching psycopg2.IntegrityError.
- **Referential Integrity (Foreign Keys):** The username columns in the wallet and subscriptions tables are strictly linked to users(username) via a FOREIGN KEY constraint. This one-to-many relationship ensures that no orphaned financial data can exist in the system.
- **Not Null:** Critical fields like card_limit, monthly_cost, and password_hash are enforced with NOT NULL constraints.

## 4.3 Procedural Design

Procedural design provides a systematic approach to developing the logic and algorithms that drive the application.

### 4.3.1 Logic Diagrams

The system's control flow is highly systematic. A user input triggers a state change in Streamlit, invoking a query string generation. This query fetches live data via the Tavily API. The retrieved data and user context are wrapped into a structured prompt and executed against the Gemini LLM. The AI's output is then intercepted by Python scripts for deterministic mathematical operations before rendering on the UI.



(Send to Google Gemini LLM)

[ OUTPUT EXTRACTION ]
(Receive JSON / Markdown from AI)

[ DETERMINISTIC PROCESSING ]
(Python Intercepts for Exact Math Calculations)

[ UI RENDERING ]
(Display Final Dashboard / Export PDF)

**4.3.2 Data Structures** The application relies heavily on dynamic data structures to manage state and logic:

- **Dictionaries (Session State):** Streamlit's st.session_state uses complex nested dictionaries to persist profile data (user_data) and cached AI responses (waiver_card, offers_card) across rapid page re-runs.
- **DataFrames:** The pandas library transforms raw tuple sets from PostgreSQL into structured DataFrame objects for the Subscription Saver engine, allowing dynamic filtering and mathematical aggregations.
- **JSON Objects:** The AI agent forces the Gemini model to output strictly formatted JSON payloads for the Penalty Audit and Point Valuation Matrix, which Python parses into native dictionaries for UI rendering.

**4.3.3 Algorithms Design** The system implements critical algorithms to ensure resilience and mathematical accuracy:

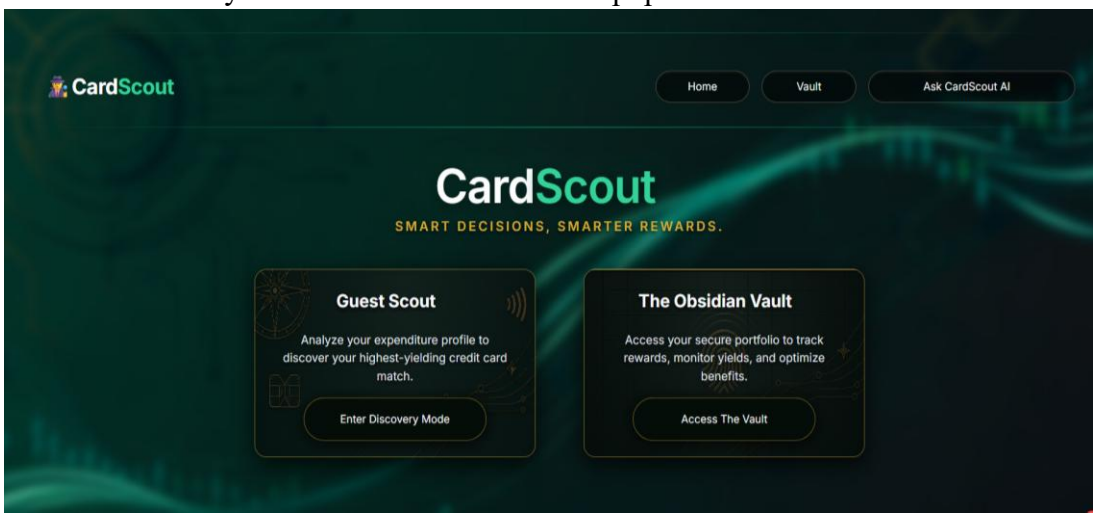
- **API Resilience and Rotation Algorithm:** To bypass strict rate-limiting, the system maintains a KEY_POOL of up to 10 Gemini API keys. The generate_with_resilience algorithm wraps API calls in a try-except block. If a Quota Error (429) occurs, the rotate_key() algorithm executes: current_key_index = (current_key_index + 1) % len(KEY_POOL), instantly switching the active key and retrying the request seamlessly.
- **Fee Waiver Utilization Algorithm:** The algorithm calculates the required monthly run-rate (req_monthly = amount_left / months_left). It then determines the credit utilization ratio (utilization = req_monthly / card_limit). If the utilization exceeds 0.30 (30%), the algorithm forces a strict "HOLD" verdict to protect the user's CIBIL score.

## 4.4 User Interface Design

The user interface was built to simulate an elite, futuristic financial command center. The UI maps user task requirements into an immersive environment utilizing custom CSS (theme.css) injected via Streamlit's st.markdown.

- **External Components:** Features a universal dark frosted-glass overlay (backdrop-filter: blur(15px)), animated SVG backgrounds (circuit boards and candlestick charts), and an extruded 3D hero title.

- **Internal Navigation:** Native Streamlit sidebars and headers are deliberately hidden using CSS display: none !important; to provide a clean, web-app feel. Navigation is handled via dynamic tabs and interactive st.popover menus.




## 4.5 Security Issues

Handling financial parameters requires strict real-time security architectures to prevent data breaches:

- **Cryptographic Password Storage:** Plain-text passwords are never stored. The system uses the Python hashlib library to implement PBKDF2-HMAC-SHA256 hashing. A cryptographically secure 16-byte random salt is generated per user, and the hashing algorithm executes 100,000 iterations to thwart brute-force and rainbow-table attacks.
- **SQL Injection Prevention:** All PostgreSQL queries inside db_manager.py utilize parameterized query execution (%s placeholders), preventing malicious users from injecting destructive SQL commands through the UI input fields.
- **Secret Management:** Sensitive API keys and database URLs are never hard-coded. They are loaded dynamically using python-dotenv and Streamlit Secrets, with regex extraction ensuring accidental trailing characters in .env files are ignored.

## 4.6 Test Cases Design

Test cases are designed to provide easy detection of errors with minimal effort across varied conditions.

- **Authentication Bounds:** Testing successful login, invalid password rejection, and the prevention of duplicate username registration.
- **API Failure Simulation:** Testing the system's ability to gracefully fall back to gemini-3.1-flash-lite if the primary gemini-3.5-flash-lite model crashes or is deprecated, and ensuring the rotate_key() function accurately loops the API pool without throwing unhandled exceptions.
- **Mathematical Edge Cases:** Ensuring that division-by-zero errors are caught in utilization algorithms (e.g., if a user inputs a 0 card limit, the system defaults the denominator to 1.0).
- **Regex Parsing Tolerance:** Ensuring that the AI parser extracts floats correctly even if the LLM outputs unpredictable text (e.g., extracting 3.5 from "The markup is 3.5% plus taxes").

# CHAPTER 5: IMPLEMENTATION AND TESTING

## 5.1 Implementation Approaches

The implementation of CardScout AI followed an iterative and modular Agile methodology. The system was developed by breaking down the overall architecture into manageable, decoupled layers: the Presentation Layer (Streamlit UI), the Data Layer (PostgreSQL database operations), the AI/Business Layer (Gemini and Tavily integrations), and the Utility Layer (PDF generation).

This separation of concerns ensured that frontend UI updates did not interfere with complex backend AI prompts or database connections. The implementation was standard-driven, adhering to PEP 8 Python coding standards, utilizing parameterized SQL queries for security, and leveraging environment variables (.env) to isolate sensitive API keys from the source code.

## 5.2 Coding Details and Code Efficiency

Rather than including the full source code, the most critical algorithmic segment driving the application's core logic is highlighted below.

**Resilient Generative AI Execution Algorithm** The generate_with_resilience() function acts as the brain of the platform. Since public AI APIs are prone to strict rate limits and quota exhaustion, this code ensures high availability by automatically pooling and rotating API keys, while also utilizing a fallback mechanism for deprecated models.

Python:

```
def generate_with_resilience(prompt):
    """Executes Gemini generation with automatic key rotation and model fallback."""
    for model_name in MODEL_FALLBACKS:
        attempts = 0
        max_attempts = max(len(KEY_POOL), 1)

        while attempts < max_attempts:
            try:
                model = get_gemini_model(model_name)
                response = model.generate_content(prompt)
                return response.text
            except Exception as e:
                err_lower = str(e).lower()
                is_quota = any(k in err_lower for k in ["429", "quota", "resource_exhausted", "rate
limit"])
                is_model_err = any(k in err_lower for k in ["404", "not found"])

                if is_model_err:
                    break # Model name unavailable, try next fallback model

                if is_quota:
                    attempts += 1
```

```
                rotate_key() # Automatically switch to the next API key
                time.sleep(1)
                continue

            raise e

    raise RuntimeError("All API keys and fallback models have reached their rate limit.")
```

*Code Explanation:* This algorithm intercepts exceptions thrown by the Google Generative AI library. If a 429 "Quota Exhausted" error occurs, it triggers the rotate_key() function to switch the active API key and retries the prompt transparently without crashing the user interface. If a 404 "Not Found" error occurs (e.g., if a model is deprecated), it dynamically shifts to the next model in the MODEL_FALLBACKS array.

**5.2.1 Code Efficiency** Code optimization was heavily prioritized to ensure low latency and minimal resource consumption:

- **Intelligent Caching:** Streamlit's @st.cache_data(ttl=3600) decorator is utilized across all AI functions. If two users ask for the fee waiver target of the "HDFC Regalia" card within an hour, the system serves the cached mathematical result instantly, completely bypassing the network call to Gemini. This drastically reduces API costs and execution time.
- **Regex over Heavy LLM Parsing:** Instead of forcing the LLM to output massive JSON structures for simple queries, lightweight Regular Expressions (re.findall(r"[-+]?\d*\.\d+|\d+", text)) are used to instantly strip out numerical data (like Forex markup floats) from the AI's textual response.
- **Connection Pooling:** Database queries execute quickly by opening and gracefully closing the psycopg2 cursor immediately after the transaction commits, preventing memory leaks and connection timeouts on the cloud server.

## 5.3 Testing Approach

Testing was conducted iteratively alongside development, following a structured methodology to ensure both functional accuracy and robust user acceptance.

**5.3.1 Unit Testing** Unit testing dealt with testing individual functions and modules in isolation.

- The hash_password(password, salt) function was tested to ensure that identical passwords with different cryptographic salts yielded entirely different hashes.
- The Regex extraction functions were fed unpredictable text (e.g., "The markup is approximately 3.5% inclusive of taxes") to ensure the system successfully extracted the float 3.5 independently of surrounding conversational filler.
- The Fee Waiver Pacing algorithm was tested with edge cases, such as passing a 0 credit limit to verify that the code gracefully handled potential division-by-zero errors.

**5.3.2 Integrated Testing** Integrated testing brought all disparate modules together into a unified testing environment to check for interoperability and data integrity.

- The end-to-end data flow was verified: A user registers via the Streamlit UI, the data routes to the PostgreSQL database, the user logs into the Obsidian Vault, adds a credit

card, and successfully executes the "Forex Engine" which correctly triggers the AI API.

- The PDF export module (export_tools.py) was tested to ensure that AI-generated Markdown roadmaps were seamlessly parsed, sanitized of non-Latin-1 characters, and rendered into a formatted FPDF document.

**5.3.3 Beta Testing** The application was deployed to the Streamlit Community Cloud for live beta testing. A select group of peers tested the application on various devices (desktops, tablets, and smartphones) across different network speeds. This phase helped identify UI layout shifts on smaller screens and highlighted the need for visible loading spinners (st.spinner) during API calls to improve user experience.

## 5.4 Modifications and Improvements

During the beta testing phase, a critical bug was identified: Streamlit's reactive architecture was causing the application to re-run the entire script every time a user typed a character into a text box. This resulted in continuous, unintended background API calls to the Gemini LLM, rapidly exhausting the API free-tier daily quota.

To improve and stabilize the system, a major architectural modification was implemented. All AI function calls inside the Obsidian Vault (such as get_utility_cashback_rate) were strictly locked behind explicit st.button triggers (e.g., "Calculate AI Yield"). Furthermore, the results were aggressively bound to st.session_state using dynamic cache keys. This modification successfully halted the API leakage, optimized the codebase, and ensured that network requests were only dispatched upon deliberate user action.

## 5.5 Test Cases

Test cases were designed to provide easy detection of errors under a variety of conditions.

| Test Case ID | Test Description | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| **TC_01** | Attempt registration with an already existing Commander ID (username). | Database throws IntegrityError; UI displays "Commander ID already exists." | Error caught; warning displayed on UI. | **Pass** |
| **TC_02** | Enter an expense amount exceeding 30% of the active card's | Co-Pilot mathematically identifies the threshold breach and returns a strict "🔴 HOLD" verdict with CIBIL warning. | System correctly issues the red HOLD warning. | **Pass** |

| Test Case ID | Test Description | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| | limit in the Co-Pilot. | | | |
| **TC_03** | Request a card roadmap with an identical profile submitted twice in a row. | The system skips the API generation loading screen and instantly serves the roadmap from st.session_state. | Cached state utilized; zero latency observed. | **Pass** |
| **TC_04** | Deplete the active Gemini API key's Request-Per-Minute (RPM) limit. | The generate_with_resilience algorithm catches the 429 error, rotates to Key #2, and completes the generation. | Automatic failover executed successfully. | **Pass** |
| **TC_05** | Download the AI-generated roadmap containing emojis and ₹ symbols. | The sanitize_for_pdf function cleans the text; FPDF generates a readable document without Unicode rendering crashes. | PDF downloads successfully with formatted tables. | **Pass** |

# CHAPTER 6: RESULTS AND DISCUSSION

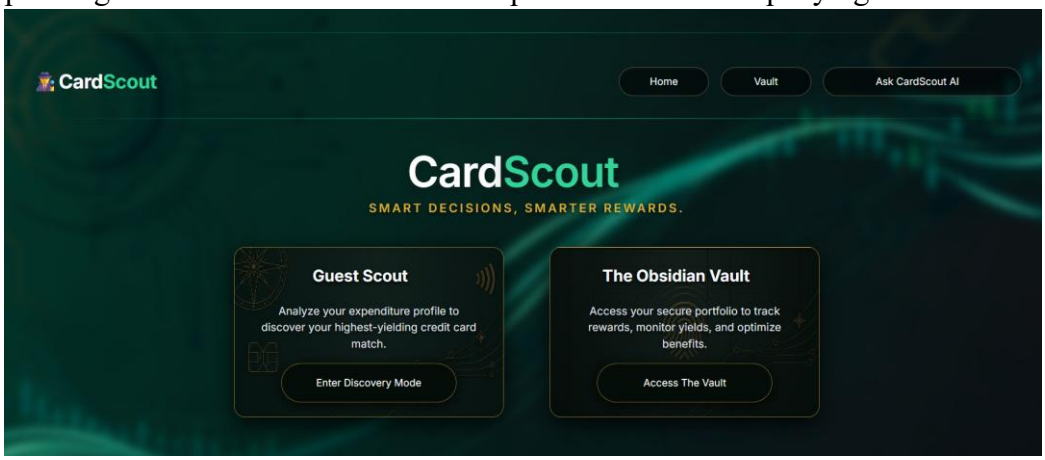
## 6.1 Test Reports

The testing phase of CardScout AI yielded highly positive results, validating both the functional architecture and the generative AI integration. The test reports, based on the test cases designed in the previous chapter, demonstrate that the application is capable of handling problematic situations—such as API rate limits and database constraints—without crashing.

- **API Resilience Validation:** During peak testing, when the primary Gemini API key exhausted its Request-Per-Minute limit, the system successfully caught the exception and rotated to secondary keys in the pool without interrupting the user's session.
- **Database Integrity Validation:** The PostgreSQL database consistently blocked duplicate Commander ID registrations and successfully maintained referential integrity. When a card was deleted from the Digital Wallet, the system accurately prevented orphaned records.
- **Mathematical Accuracy:** The deterministic Python algorithms performed flawlessly under varied sample inputs. For example, testing the Forex Engine with a target budget of $2,500 at a 3.5% markup correctly calculated the INR penalty, actively appending the mandatory 18% Indian GST for exact precision.

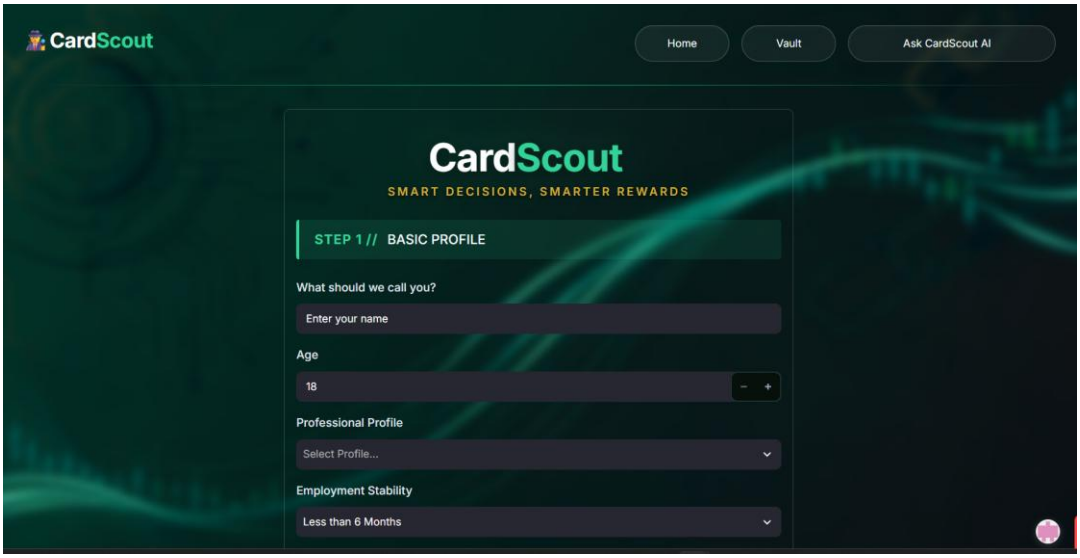
## 6.2 User Documentation

This section serves as a comprehensive user manual, defining the working of the software and explaining its different functions and components with accompanying screenshots.



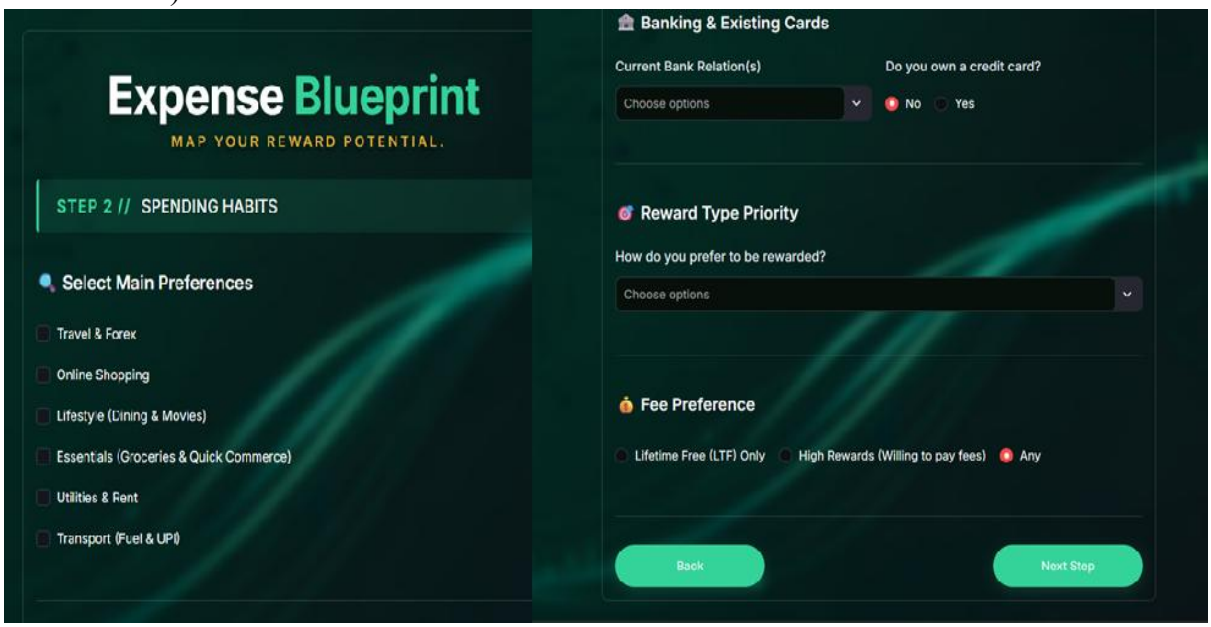

### 6.2.1 Guest Scout – Basic Profile (Step 1)

The Guest Scout module begins with a profiling wizard. Users input their name, age, professional profile, employment stability, and estimated credit score band. This demographic data forms the baseline for the AI's financial calculations.

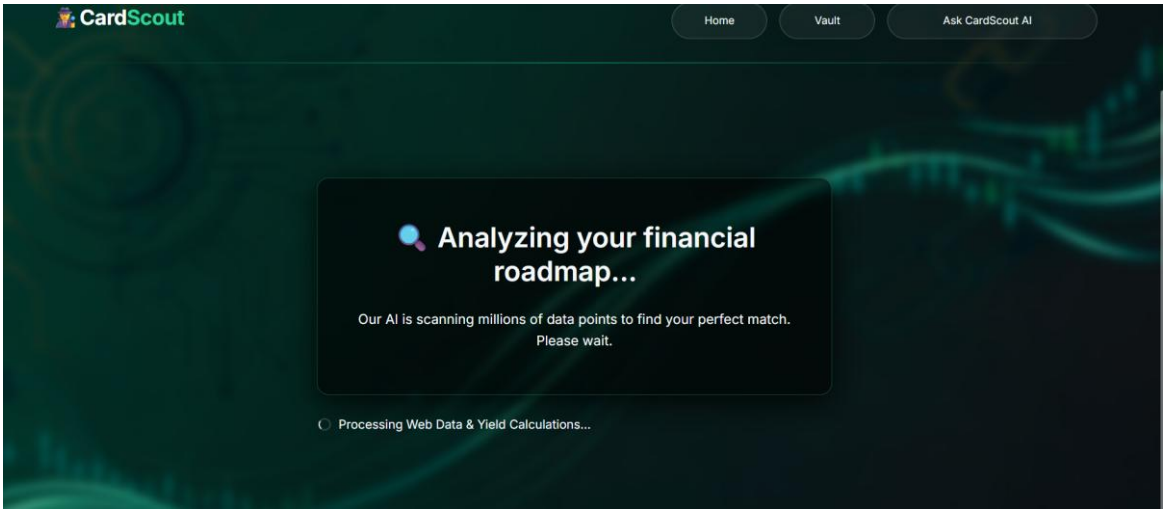


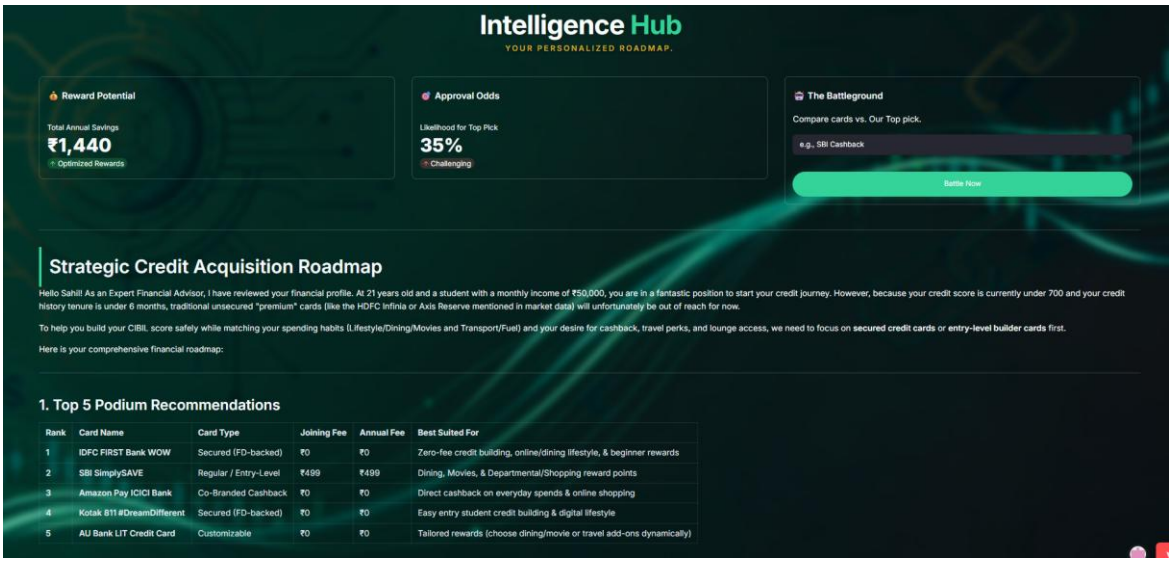

### 6.2.2 Expense Blueprint (Step 2)

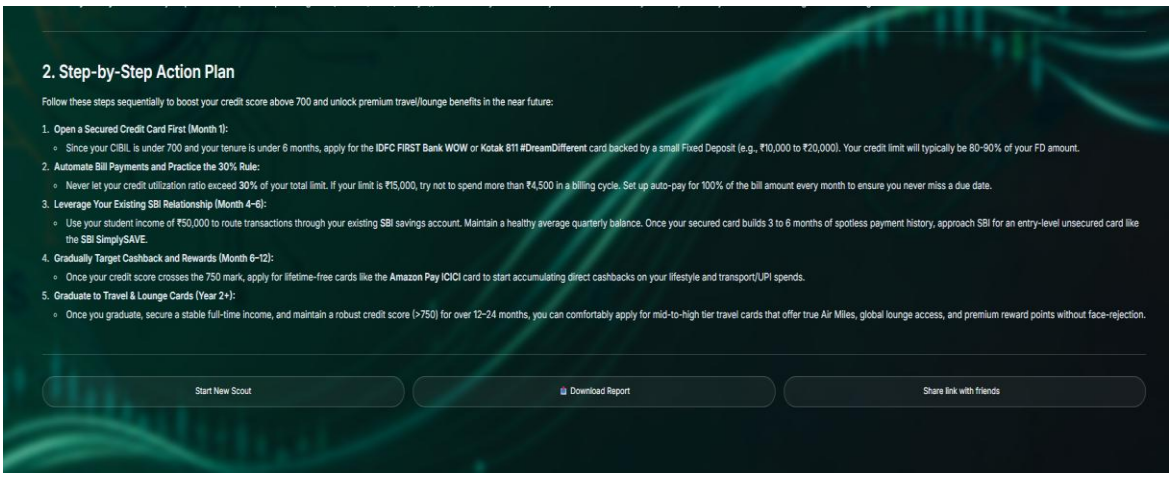
Users map their exact monthly spending habits across specific verticals (such as Travel, Quick Commerce, and Utilities) and select their preferred reward types (e.g., Direct Cashback vs. Air Miles).



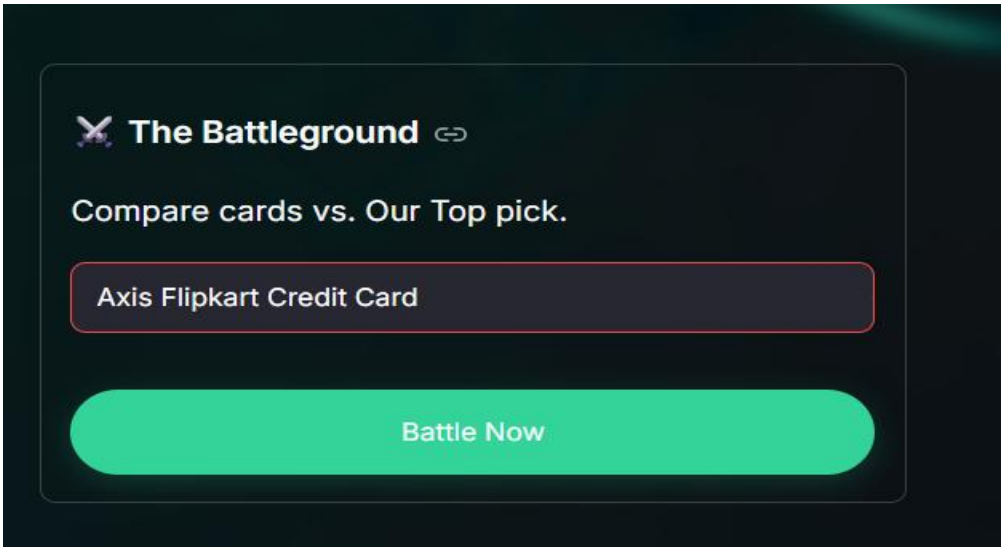

**6.2.3 Intelligence Hub** Upon completing the wizard, the AI synthesizes the data and generates the Intelligence Hub. This dashboard displays the user's Reward Potential (estimated annual savings) and Approval Odds. Below this, the "Strategic Credit Acquisition Roadmap" provides a clean, Markdown-formatted Top-5 card recommendation.

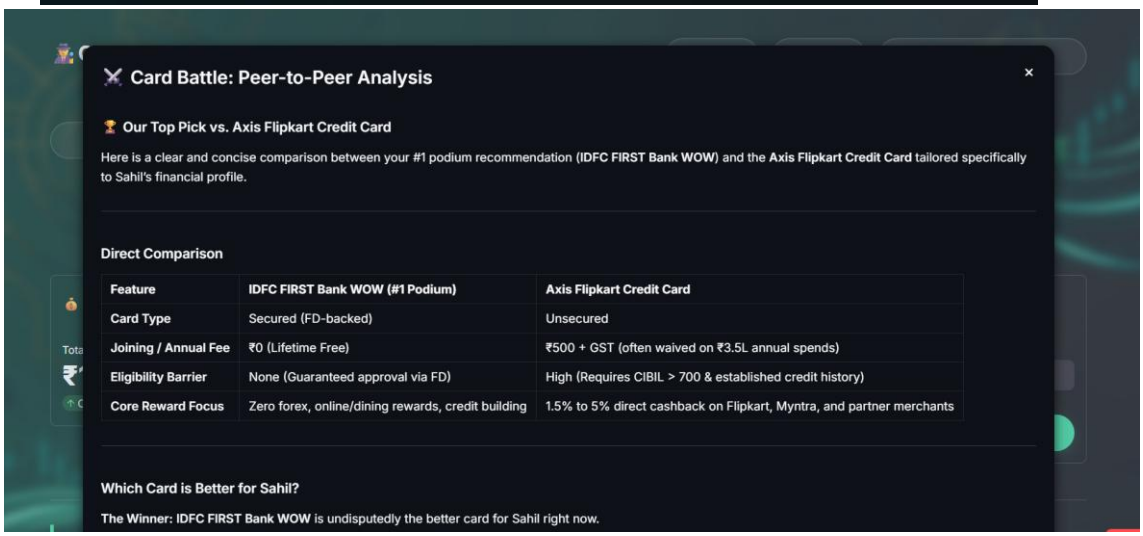


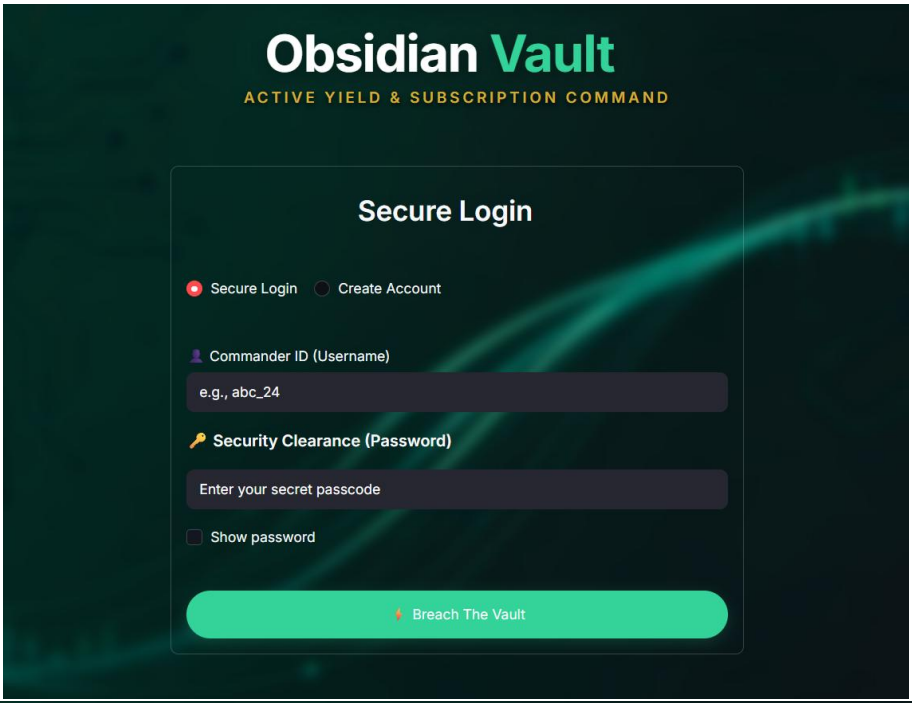



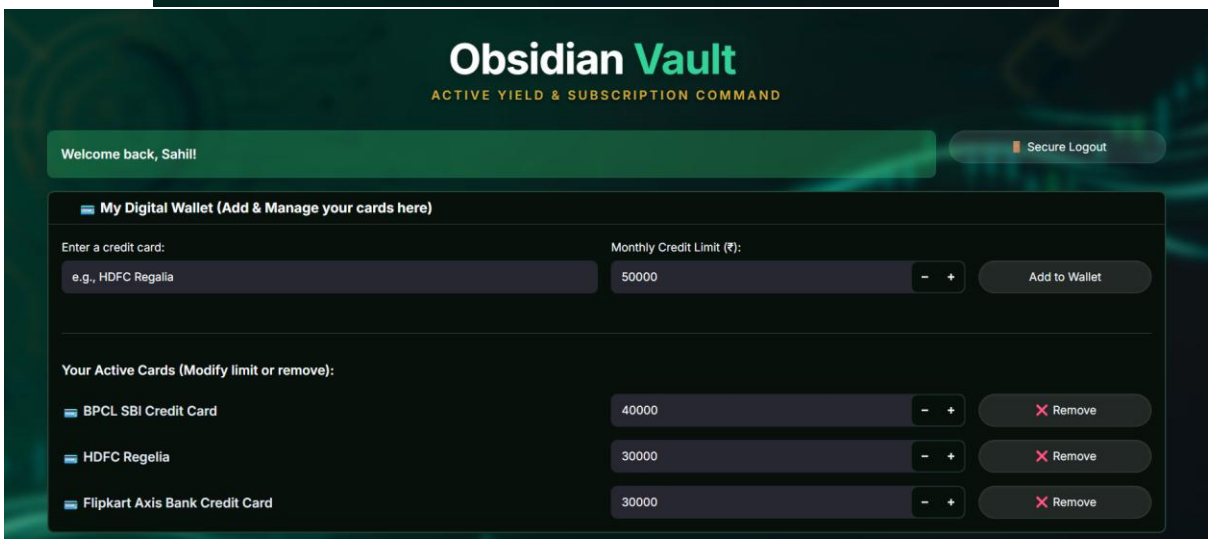

**6.2.4 The Battleground** A peer-to-peer analysis tool located within the Intelligence Hub. Users can input any credit card name, and the AI evaluates it directly against the system's #1 recommended card, explaining clearly which card is mathematically better for the specific user.



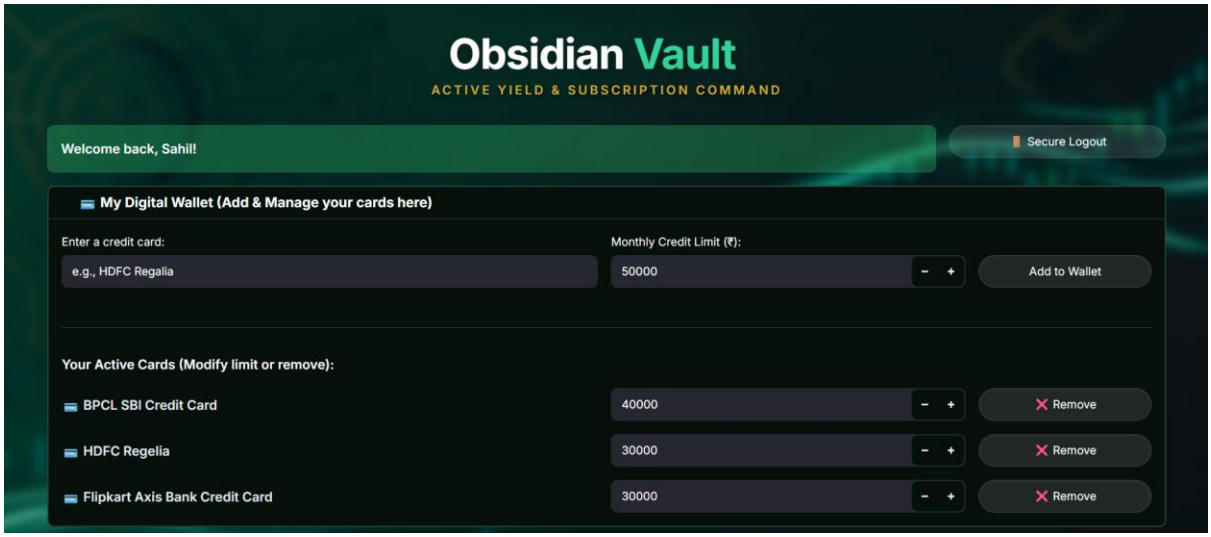




**6.2.5 Obsidian Vault – Login & Digital Wallet** Users can initialize a "Commander Profile" to securely access the Obsidian Vault. Once authenticated, the Digital Wallet allows users to add their existing credit cards and monthly credit limits to the PostgreSQL database.

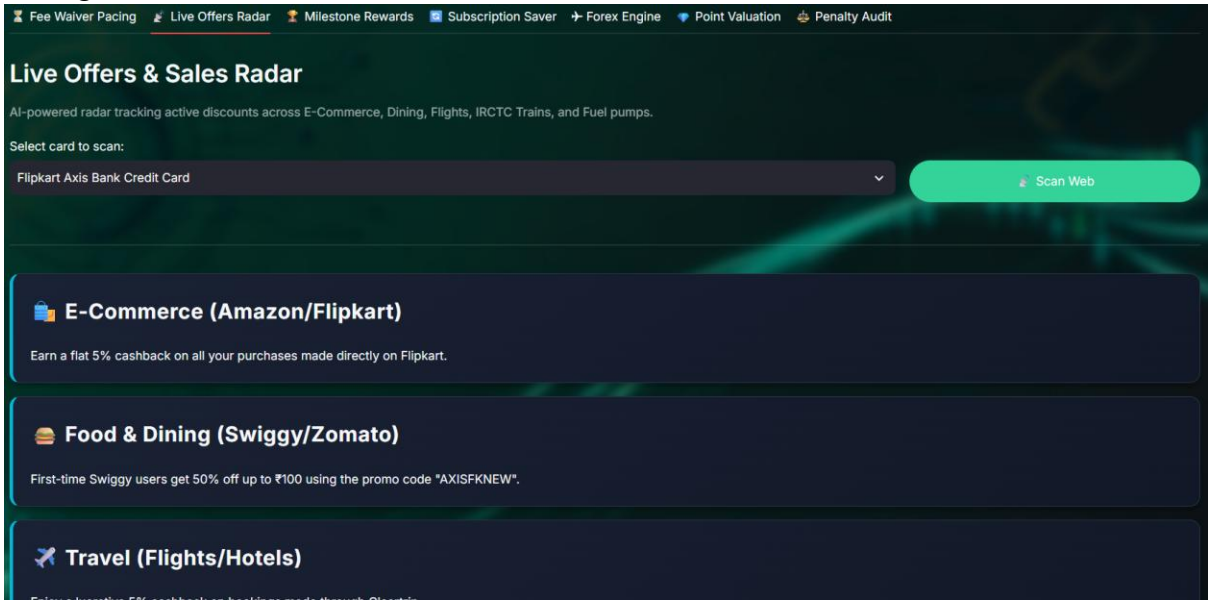


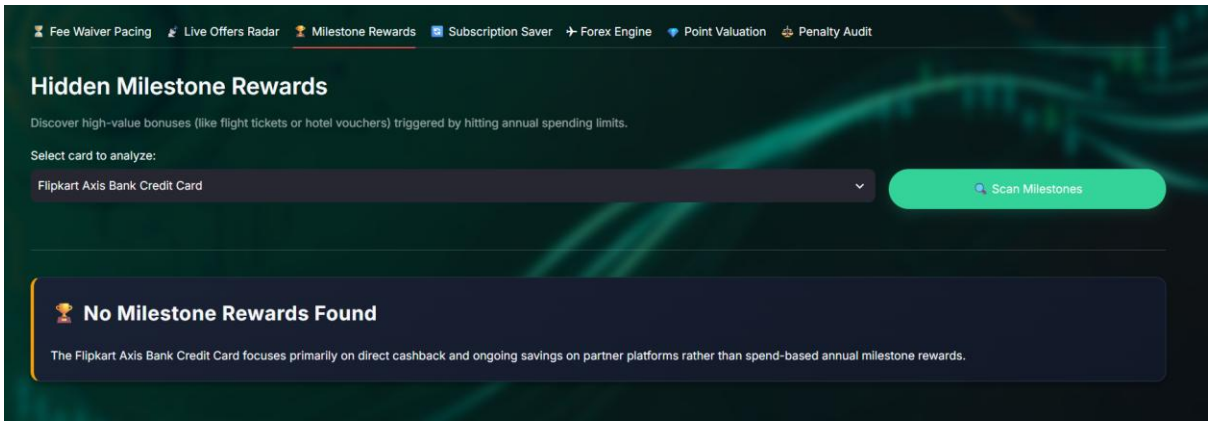

**6.2.6 Fee Waiver Pacing** This engine actively tracks if a user is spending enough to qualify for a card's annual fee waiver. The system calculates the required monthly run-rate and strictly evaluates it against the user's saved credit limit. If chasing the waiver requires over 30% utilization, the system issues a red "HOLD" verdict to protect the user's CIBIL score.



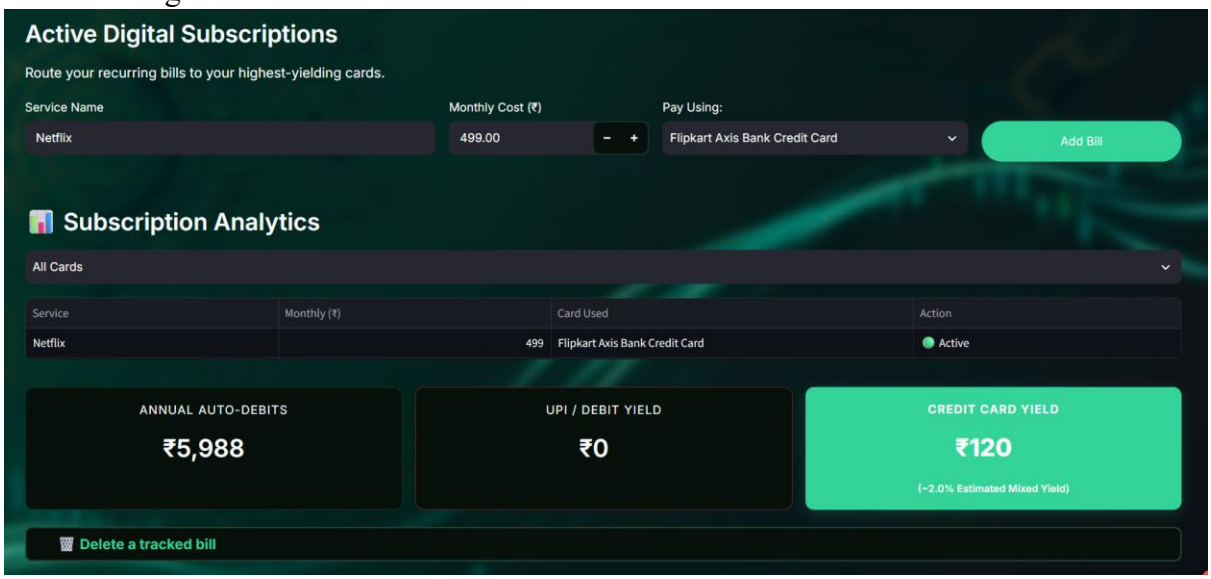

**6.2.7 Live Offers Radar** Replacing static affiliate websites, this tool performs real-time web scraping via the Tavily API to locate active discounts across five categories: E-Commerce, Dining, Travel, IRCTC, and Fuel.



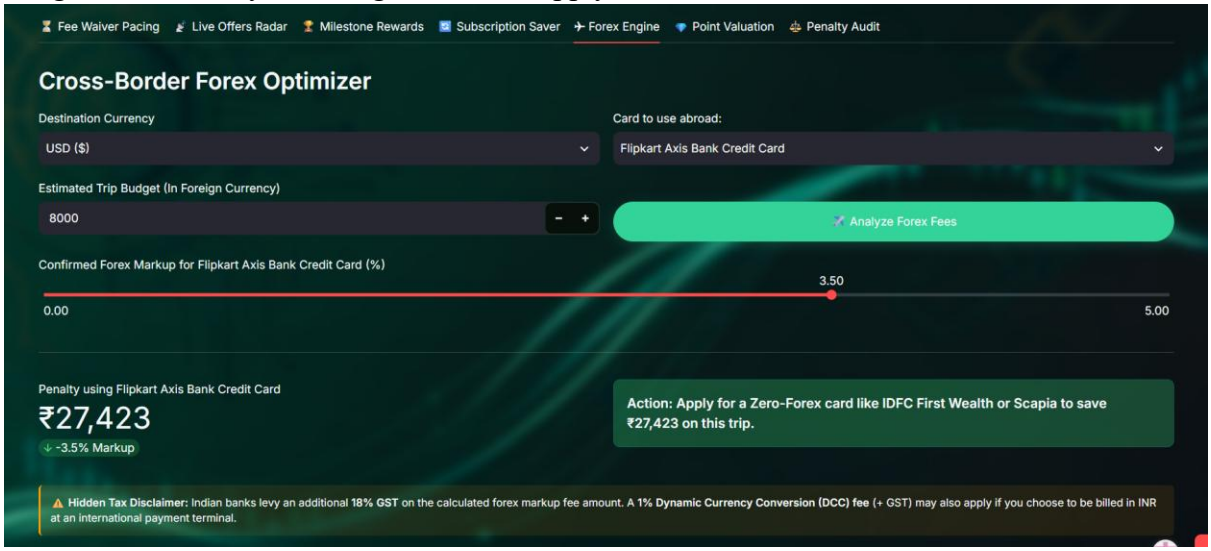

**6.2.8 Milestone Rewards** This engine extracts hidden, spend-based milestone rewards (such as free flight tickets) from complex bank terms and displays them as gamified, gold achievement cards.



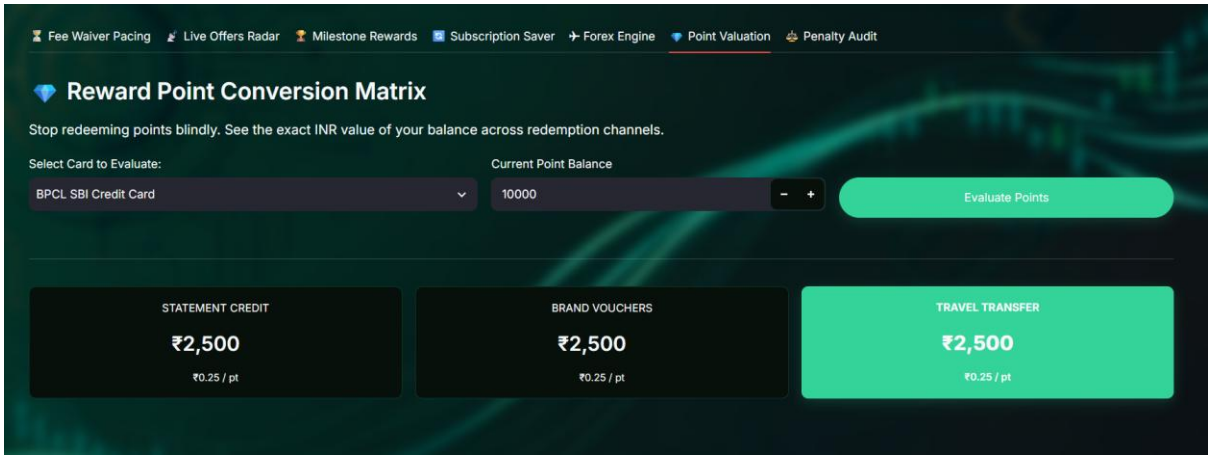
**6.2.9 Subscription Saver** Users log their recurring digital bills (e.g., Netflix, Rent). The AI fetches the specific cashback percentage the chosen card yields for that merchant category and compares it against standard UPI payments, dynamically displaying the user's automated annual savings.



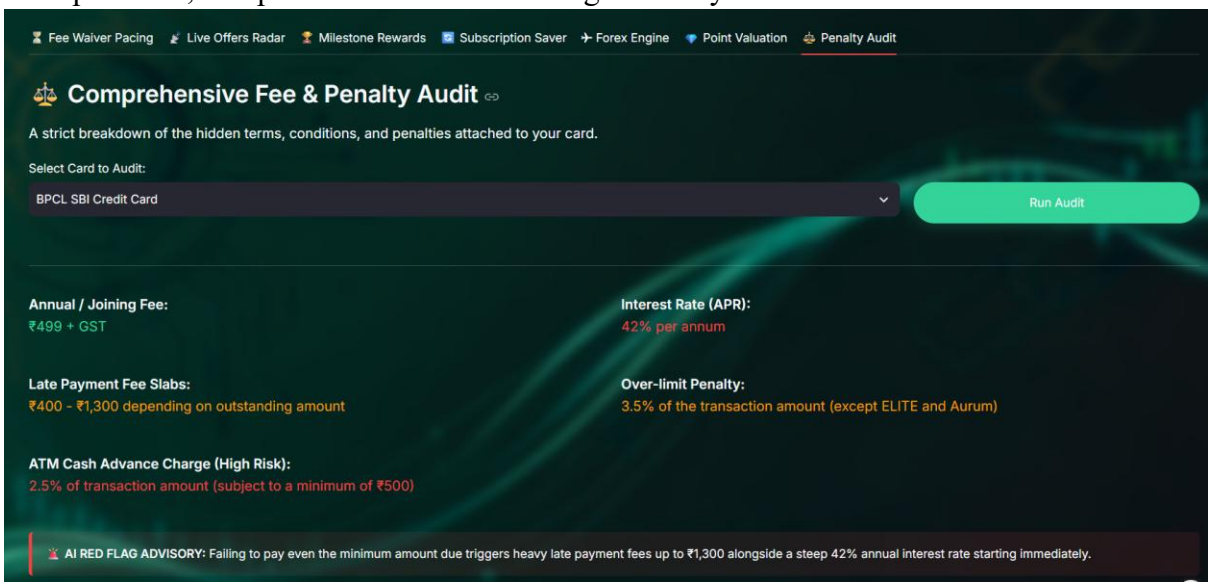

**6.2.10 Forex Engine** This cross-border optimizer evaluates hidden international transaction costs. It calculates the total penalty (base markup + 18% GST) based on the user's travel budget and currency, advising whether to apply for a Zero-Forex card.




**6.2.11 Point Valuation Matrix** This tool solves "blind redemption" by fetching the exact INR value of a single reward point across Statement Credit, Brand Vouchers, and Travel Miles. It identifies arbitrage opportunities, warning users if they are losing money by choosing cash over travel miles.




**6.2.12 Penalty Audit** The AI performs a forensic extraction of a card's terms and conditions, generating a structured JSON audit of Annual Percentage Rates (APR), Late Fees, and Over-limit penalties, complete with an AI "Red Flag Advisory".




**6.2.13 PDF Report Export** A one-click export feature that sanitizes the AI-generated Markdown roadmap of unsupported Unicode characters and dynamically renders a sleek, professional PDF report using the FPDF library.

# CardScout AI: Strategic Card Roadmap

*Personalized Yield & Optimization Report for Sahil*

Hello Sahil! As an Expert Financial Advisor, I have reviewed your financial profile. At 21 years old and a student with a monthly income of Rs. 50,000, you are in a fantastic position to start your credit journey. However, because your credit score is currently under 700 and your credit history tenure is under 6 months, traditional unsecured "premium" cards (like the HDFC Infinia or Axis Reserve mentioned in market data) will unfortunately be out of reach for now.

To help you build your CIBIL score safely while matching your spending habits (Lifestyle/Dining/Movies and Transport/Fuel) and your desire for cashback, travel perks, and lounge access, we need to focus on secured credit cards or entry-level builder cards first.

Here is your comprehensive financial roadmap:

## 1. Top 5 Podium Recommendations

**1. IDFC FIRST Bank WOW**
Card Type: Secured (FD-backed) | Joining Fee: Rs. 0 | Annual Fee: Rs. 0 | Best Suited For: Zero-fee credit building, online/dining lifestyle, & beginner rewards

**2. SBI SimplySAVE**
Card Type: Regular / Entry-Level | Joining Fee: Rs. 499 | Annual Fee: Rs. 499 | Best Suited For: Dining, Movies, & Departmental/Shopping reward points

**3. Amazon Pay ICICI Bank**
Card Type: Co-Branded Cashback | Joining Fee: Rs. 0 | Annual Fee: Rs. 0 | Best Suited For: Direct cashback on everyday spends & online shopping

**4. Kotak 811 DreamDifferent**
Card Type: Secured (FD-backed) | Joining Fee: Rs. 0 | Annual Fee: Rs. 0 | Best Suited For: Easy entry student credit building & digital lifestyle

**5. AU Bank LIT Credit Card**

# CHAPTER 7: CONCLUSIONS

## 7.1 Conclusion

CardScout AI successfully demonstrates that combining real-time web scraping, a Large Language Model, and deterministic financial calculations can deliver highly personalized, actionable credit card advice that traditional, static comparison websites cannot match. The project effectively covers the complete consumer financial journey: from discovering suitable cards and comparing alternatives to sophisticated post-acquisition management.

Through the Obsidian Vault, the system empowers users to actively track fee waivers, uncover live offers and hidden milestones, optimize recurring subscriptions, avoid forex markup traps, mathematically value reward points, and audit dangerous penalty clauses. Security was rigorously addressed using salted PBKDF2 password hashing and parameterized SQL queries, while system reliability was guaranteed through intelligent API key rotation, model fallback protocols, and query caching.

### 7.1.1 Significance of the System

The primary significance of CardScout AI lies in its ability to bridge the massive gap between complex, predatory banking products and everyday consumer financial literacy. By automating complex financial analysis, the system democratizes elite-level advisory services, allowing everyday users to maximize their financial returns while actively protecting their CIBIL scores from high credit utilization. It sets a new standard for how Generative AI can be applied responsibly in the FinTech sector by pairing LLM text extraction with strict, hard-coded arithmetic.

## 7.2 Limitations of the System

While CardScout AI is highly functional, certain limitations were encountered during testing and development that users must consider:

- **Dependency on Search Quality:** The AI's responses are heavily dependent on the quality and accuracy of the live web data retrieved by the Tavily Search API. Therefore, users are advised to verify critical terms with the issuing bank before making final financial commitments.
- **Approximated Metrics:** The Reward Potential savings figure is a mathematical estimate (calculated generically as 3% of monthly spending), and the Approval Odds are indicative based on basic credit score bands rather than a full credit pull.
- **Static Forex Rates:** In the current version of the application, the currency conversion rates used in the Forex Engine are fixed approximations rather than live, real-time market trading rates.

## 7.3 Future Scope of the Project

The following are new areas of investigation and features planned for future iterations of CardScout AI, which were not completed in the current build due to time constraints:

- **Live Exchange Rates:** Integrating a financial API to fetch live, real-time currency exchange rates for the Forex Engine to ensure pinpoint accuracy.

- **Automated Spend Tracking:** Adding a transaction import feature (via CSV upload or India's Account Aggregator framework) to track user spending automatically, removing the need for manual inputs.
- **Push Notifications:** Implementing an alert system to send users reminders for fee-waiver deadlines, statement due dates, and expiring coupon offers.
- **Advanced CIBIL Integration:** Adding a secure integration to pull actual credit scores for a more data-driven and precise approval-probability model.
- **Conversational Memory:** Upgrading the CardScout Co-Pilot into a full conversational chat interface with session memory for continuous, multi-turn financial planning.
- **Accessibility and Security Upgrades:** Adding multi-language support (Hindi, Marathi), creating a mobile-optimized Progressive Web App (PWA), and introducing Two-Factor Authentication (2FA) with automated password resets.

# REFERENCES


[1] Streamlit. (n.d.). *Streamlit Documentation*. Retrieved from https://docs.streamlit.io

[2] Google. (n.d.). *Google Generative AI (Gemini) API Documentation*. Retrieved from https://ai.google.dev/docs

[3] Tavily. (n.d.). *Tavily Search API Documentation*. Retrieved from https://docs.tavily.com

[4] Supabase. (n.d.). *Supabase Documentation*. Retrieved from https://supabase.com/docs

[5] Psycopg. (n.d.). *Psycopg2 Documentation*. Retrieved from https://www.psycopg.org/docs/

[6] Python Software Foundation. (n.d.). *hashlib — Secure hashes and message digests*. Python 3.11 Documentation. Retrieved from https://docs.python.org/3/library/hashlib.html

[7] PyFPDF. (n.d.). *fpdf2 Documentation*. Retrieved from https://py-pdf.github.io/fpdf2/